



Recovery en Bases de Datos Relacionales

Write-Ahead Logging (WAL) y su implementación en PostgreSQL y MySQL



Qué vamos a ver hoy

01

El problema

Por qué el buffer pool pone en riesgo Atomicity y Durability



02

Write-Ahead Logging

La regla de oro y la anatomía de un log record



03

ARIES

Analysis · Redo · Undo — el algoritmo detrás de casi todos los motores



04

PostgreSQL

WAL, checkpoints, replicación y PITR



05

MySQL / InnoDB

Redo log, undo log, doublewrite buffer y binlog



06

Comparativa

Dos filosofías de recovery, cara a cara



El buffer pool y el riesgo para ACID

Escribir cada cambio directamente a disco sería demasiado costoso: las páginas se modifican en RAM (buffer pool) y su escritura a disco se posterga.



Durability (D)

Si hubo COMMIT, el cambio debe sobrevivir a un crash — aunque la página aún no esté en disco.



Atomicity (A)

Si la transacción no llegó a COMMIT, sus efectos parciales deben poder deshacerse por completo.



RAM — Buffer Pool

Páginas modificadas ("dirty pages") esperando para bajar a disco. Rápido de modificar, volátil.



Disco — Log + Datos

El log se escribe primero, en forma secuencial (append-only) — barato. Las páginas de datos bajan después, en su propio tiempo.

La regla de oro del WAL

Antes de escribir una página modificada a disco → hay que escribir primero al log, en disco, el registro que describe ese cambio.

Y antes de confirmar un COMMIT al cliente, el log de esa transacción tiene que estar físicamente en disco (fsync).



1. Modificar

La transacción cambia una página en el buffer pool (RAM).



2. Loguear

Se genera el log record con el cambio (LSN, página, valor viejo/nuevo).



3. fsync()

El log record se fuerza a disco antes de responder el COMMIT.



4. Confirmar

Recién ahí el motor le dice al cliente que el COMMIT fue exitoso.

Anatomía de un log record

Cada registro del log identifica de forma unívoca un cambio, y cada página recuerda cuál fue el último cambio que ya tiene aplicado.

LSN

Log Sequence Number: identificador único y creciente del registro.

TxID

Transacción que generó el cambio.

Página / offset

Qué página (y dentro de ella, qué parte) fue modificada.

Valor viejo

Necesario para poder hacer undo (rollback).

Valor nuevo

Necesario para poder hacer redo tras un crash.



PageLSN

Cada página en el buffer pool guarda el LSN del último cambio que ya se le aplicó.

Así, al recuperar, el motor sabe exactamente qué registros del log ya están reflejados en la página y cuáles todavía no.

→ **evita redo innecesario**

ARIES: el algoritmo detrás del recovery

Algorithm for Recovery and Isolation Exploiting Semantics — base (con variantes) de la mayoría de los motores reales.



1. Analysis

Recorre el log desde el último checkpoint para reconstruir qué transacciones estaban activas y qué páginas estaban sucias.



2. Redo

Rehace TODOS los cambios logueados desde ese punto — incluso de transacciones que después abortaron — hasta llegar al estado exacto previo al crash.



3. Undo

Deshace los cambios de las transacciones que no habían hecho COMMIT, usando los valores viejos guardados en el log.

Checkpoints acotan cuánto log hay que revisar al reiniciar: en un checkpoint se fuerza a disco cierto estado del buffer pool / log para no tener que analizar desde el principio de los tiempos.

DE LA TEORÍA A LA PRÁCTICA

Dos motores, dos filosofías de recovery



PostgreSQL — WAL nativo



MySQL / InnoDB — ARIES clásico

El WAL de PostgreSQL

El log se llama WAL directamente: archivos de 16MB en ***pg_wal/*** (antes *pg_xlog/*). Cada registro tiene un LSN. Es redo físico de páginas, con algunas optimizaciones lógicas.



Sin undo log separado: gracias a MVCC, las versiones viejas de las tuplas ya están en la tabla. Para abortar, simplemente no se aplican los cambios — no hace falta "deshacer" escribiendo encima.

wal_level

minimal / replica / logical — cuánta info se logea

checkpoint_timeout

cada cuánto se dispara un checkpoint

synchronous_commit

si el COMMIT espera el fsync del WAL

Al reiniciar tras un crash: redo desde el último checkpoint. No hay fase de undo explícita como en ARIES — MVCC hace ese trabajo "gratis".

El WAL es más que recovery

El mismo mecanismo que garantiza durability se reutiliza para otras tres cosas clave:



Replicación streaming

Los registros de WAL se envían a réplicas y se re-aplican allá — el mismo mecanismo de redo.



PITR

Point-In-Time Recovery: pg_basebackup + archivado continuo de WAL para volver a cualquier instante.



Replicación lógica

Decodificación del WAL a nivel fila (logical decoding), para replicar hacia otros sistemas.

Idea clave: recovery, replicación y PITR no son mecanismos distintos — son tres formas de reproducir la misma secuencia de cambios físicos.

InnoDB: más de un log, cada uno con su rol

A diferencia de Postgres, InnoDB usa varios mecanismos separados — es la parte que más confunde en el examen.



Redo log

`ib_logfile` / `#innodb_redo`

El WAL propiamente dicho: cambios físicos a páginas, en buffer circular de tamaño fijo. Usado en crash recovery para redo.



Undo log

`tablespaces de undo`

Valores anteriores de las filas. Sirve para rollback y para dar snapshots MVCC a lecturas consistentes.



Doublewrite buffer

`área contigua previa`

Protege contra partial page writes: escribe la página primero en un área auxiliar antes que en su lugar final.



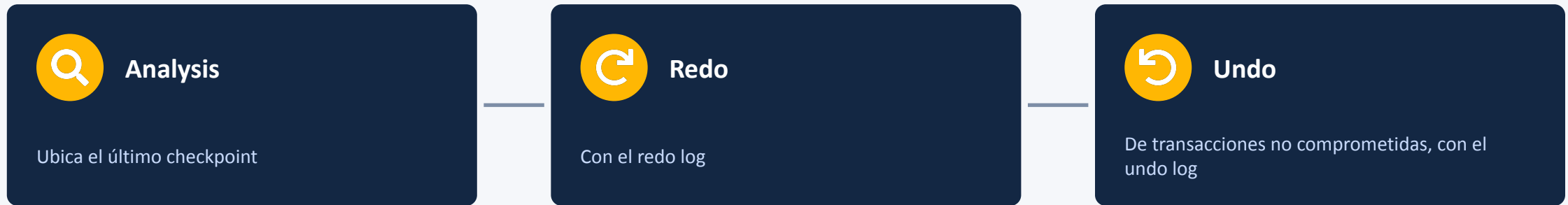
Binary log (binlog)

`log lógico, separado`

NO es el mecanismo de crash recovery. Es para replicación y PITR — statement o row-based.

Recovery en InnoDB: ARIES completo

InnoDB hace checkpoint difuso (fuzzy checkpointing) de forma continua. Al reiniciar tras un crash:



Gotcha clásico de examen

Redo log ≠ Binlog. El redo log es el WAL físico usado en crash recovery. El binlog es un log lógico separado, usado para replicación / PITR. Mantener ambos consistentes requiere un **two-phase commit interno** entre redo log y binlog — algo que Postgres no necesita, porque tiene un único WAL.

PostgreSQL vs. MySQL (InnoDB), cara a cara

	 PostgreSQL	 MySQL / InnoDB
Log físico principal	WAL (pg_wal/)	Redo log (ib_logfile)
Deshacer Tx abortadas	MVCC — sin log de undo separado	Undo log dedicado
Log para replicación / PITR	El mismo WAL	Binlog (log lógico separado)
Protección partial writes	full_page_writes	Doublewrite buffer
Algoritmo de recovery	Redo-only (variante, gracias a MVCC)	ARIES completo (analysis–redo–undo)

Tres ideas para no olvidar



El log siempre va primero

WAL no es un detalle de implementación: es la razón por la cual el buffer pool puede ser volátil sin romper durability.



Un log puede servir para todo

Postgres reutiliza el mismo WAL para recovery, replicación y PITR. Diseñar bien el log de antemano paga dividendos.



Undo no siempre es un log

MVCC (Postgres) y undo logs (InnoDB) son dos formas válidas de resolver Atomicity — no hay una única receta correcta.



¿Preguntas?

Bases de Datos II · ITBA — Recovery y Write-Ahead Logging

